however, we would store each element three times: twice as a key of each map and once as a root of some tree (this last entry could store, of course, some keys several times and some others not once).

By using this extra wrapper and a single "info" map, we make sure to store elements only once as keys.

While objects would be stored as reference, with minimal overhead, immutable values, and especially strings, would be stored by value. Therefore, even avoiding to store each element once more can lead to consistent memory saving.

We could, in theory, do even better: by storing each element in objects wrappers and using those wrappers as keys. This way, we would only store each key once, and use wrappers' references all the time, both as keys for our map(s) and as values.

Is the overhead and increased complexity of the wrapper solution worth? This depends on the assumptions you can make on the type and size of the keys. In most cases, it is probably not, so be sure to properly profile your application and analyze your input, before embarking in such optimizations.

To go back to our implementation: once again, changes are minimal. In both constructor and `add`, we just need to update the very last line:

```
parentsMap[elem] = new Info(elem)
```